Pontificia Universidad Católica Madre y Maestra

Asignatura:

Programación Web Avanzada

Profesor:

Carlos A. Camacho G.

Asignación:

Práctica 3 - Uso Docker y Docker Compose

Git:

https://github.com/firnex360/st-icc-354-001-ferb0001/tree/main/practica-3

Nombres & Matrículas:

María I. Tiburcio A.	*10150779*
Fernando E- Rodríguez B.	*10149894*

Fecha de Entrega:

2 de Marzo, 2026

República Dominicana, Santiago de los Caballeros

1. Uso de la Aplicación

Para bajar la imagen desde Docker Hub (solo la de la app): docker pull firnex/clon-mocky:latest

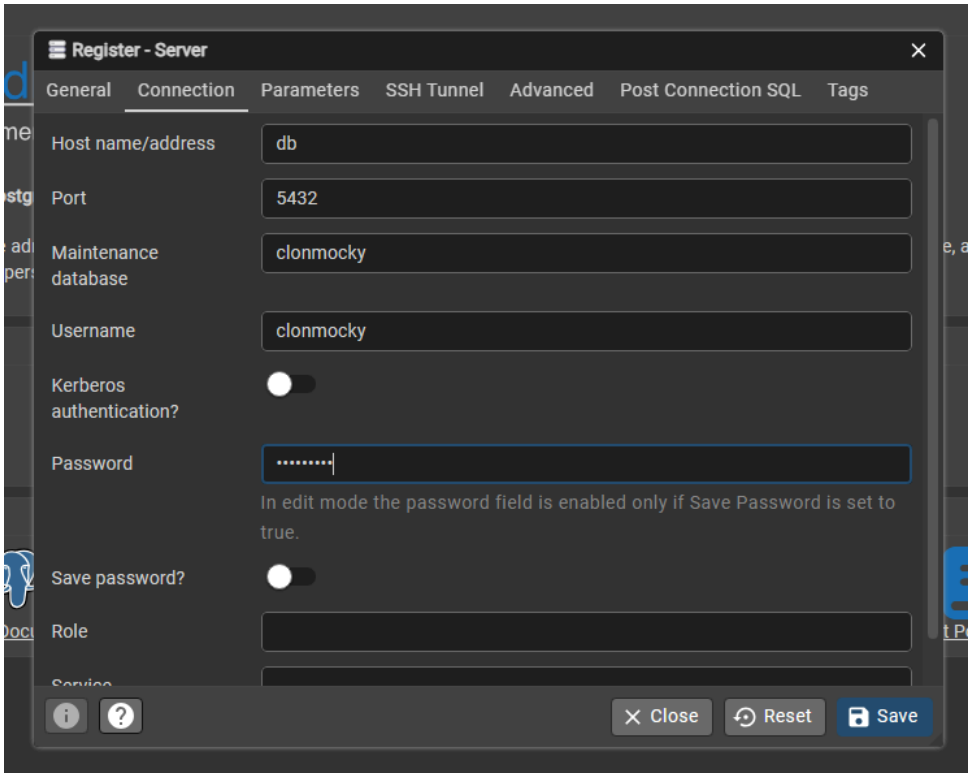
Para correr el Docker Compose: dirigirse a la carpeta practica-3 desde la terminal (hay una copia del .env y el docker-compose en la carpeta practica-3 como dentro del proyecto clon-mocky) y correr “docker compose up -d”. Este instalará todas las imágenes necesarias e iniciará el contenedor.

Por default está bajo los puertos establecidos por práctica (8080 para la app y 8082 para el panel de la DB).

Para acceder a las interfaces: Para la interfaz del app se usa user: admin y password: admin

Para la interfaz de la base de datos:
User: admin@admin.com
password : admin

Agregar base de datos al interfaz pgAdmin:



Password: clonmocky

2. Introduccion

El presente informe detalla la integración de tecnologías de contenerización para el proyecto "Mockup API Server - Spring Boot", también conocido como Clon Mocky. Con el objetivo de optimizar el desarrollo e ir preparando la aplicación para las nuevas tendencias en arquitectura de software, se procedió a implementar contenedores para separar los distintos servicios que componen el sistema.

Los objetivos principales de esta implementación fueron: implementar Docker como contenedor de software, administrar un escenario de varios contenedores (Aplicación, Base de Datos y Cliente Web), configurar redes y volúmenes internos, y automatizar el levantamiento del entorno mediante Docker Compose y variables de ambiente.

3. Desarrollo

Para cumplir con los requerimientos establecidos, se diseñó una arquitectura de tres contenedores orquestados mediante un archivo `docker-compose.yml` y un archivo de variables de entorno (`.env`).

3.1. Contenerización de la Aplicación (Dockerfile Multi-stage) Se creó un archivo `Dockerfile` para la aplicación Spring Boot implementando el patrón *multi-stage*.

- **Stage 1 (Build):** Se utilizó la imagen `eclipse-temurin:25-jdk` para disponer de las herramientas de compilación. Aquí se descargan las dependencias vía Gradle y se construye el archivo JAR ejecutable de manera aislada.
- **Stage 2 (Runtime):** Para la ejecución, se utilizó la imagen más ligera `eclipse-temurin:25-jre`. Esta etapa toma únicamente el JAR generado en la primera fase, logrando que la imagen final sea mucho más pequeña, segura y eficiente.

3.2. Base de Datos y Persistencia Para el motor de base de datos se utilizó la imagen oficial de PostgreSQL (`postgres:17-alpine`). Cumpliendo con las restricciones de seguridad arquitectónica, este contenedor **no expone** ningún puerto al entorno *host* de la máquina local. Toda la comunicación hacia la base de datos se realiza internamente a través de la red personalizada de Docker.

Para garantizar que los registros no se pierdan al detener el contenedor, se configuró un volumen de datos local mapeado a la ruta `/var/lib/postgresql/data` dentro del contenedor. La ruta de este volumen en el host está parametrizada mediante la variable `${DB_DATA_PATH:-./pgdata}`.

3.3. Cliente de Administración Web Dado que la base de datos está aislada del host, se incluyó un contenedor con la imagen oficial de `dpage/pgadmin4` para actuar como interfaz gráfica de administración de PostgreSQL. El puerto de acceso a este servicio desde el navegador fue parametrizado a través de la variable `${PGADMIN_PORT}`, utilizando por defecto el puerto `8082` tal como fue requerido.

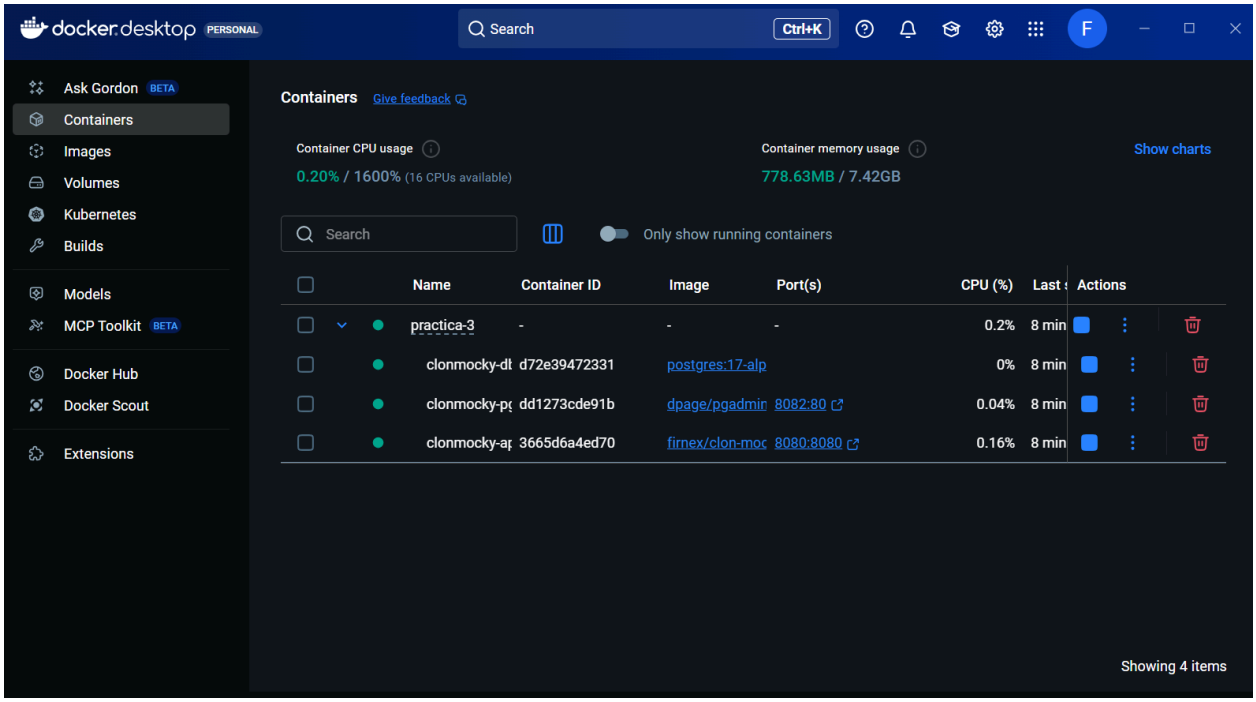
3.4. Orquestación y Parametrización Todo el ecosistema se levanta con un solo comando gracias a Docker Compose. Se configuraron los siguientes aspectos clave:

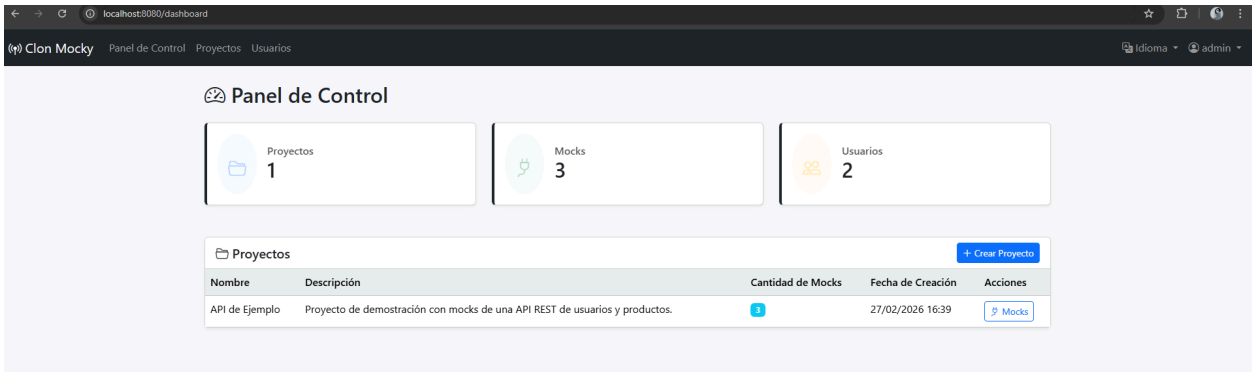
- **Red Personalizada:** Se creó la red `clon-mocky-net` tipo *bridge*, garantizando que la aplicación, la base de datos y PgAdmin puedan comunicarse de forma segura.
- **Variables de Entorno (.env):** Toda la información sensible y de conexión (URLs, credenciales de base de datos, puertos del host, usuario de Docker Hub y secretos JWT) fue abstraída al archivo `.env`. El puerto de la aplicación quedó correctamente parametrizado con el valor por defecto `8080`.
- **Healthchecks:** Se implementó una verificación de estado (`healthcheck`) usando `pg_isready` en la base de datos. La aplicación y PgAdmin utilizan la directiva `depends_on: condition: service_healthy` para asegurar que no intenten conectarse antes de que PostgreSQL esté completamente iniciado y listo.

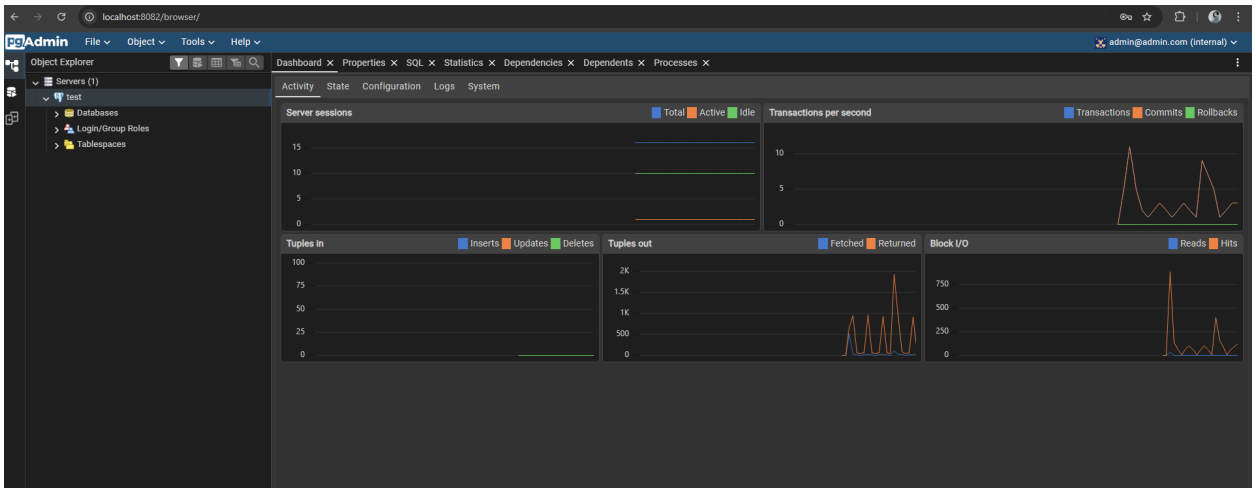
4. Conclusion

La inclusión de Docker y Docker Compose en el proyecto Clon Mocky fue un éxito rotundo. Se logró parametrizar al 100% la información de conexión y los puertos expuestos, lo que permite que cualquier desarrollador pueda levantar el proyecto en su máquina local sin conflictos de puertos ni necesidad de instalar bases de datos externas.

El uso de un `Dockerfile` multi-stage optimiza el consumo de recursos de la aplicación empaquetada. Además, al integrar la variable `${DOCKERHUB_USER}`, el proyecto está totalmente preparado para compilar sus imágenes y publicarlas en Docker Hub, cumpliendo cabalmente con todos los objetivos planteados para la actualización arquitectónica del sistema.


docker desktop PERSONAL
Search Ctrl+K
Ask Gordon BETA
Containers
Images
Volumes
Kubernetes
Builds
Models
MCP Toolkit BETA
Docker Hub
Docker Scout
Extensions
Containers Give feedback
Container CPU usage
0.20% / 1600% (16 CPUs available)
Container memory usage
778.63MB / 7.42GB
Show charts
Search
Only show running containers
Name Container ID Image Port(s) CPU (%) Last Actions
practica-3 - - - 0.2% 8 min
clonmocky-dt d72e39472331 postgres:17-alp 0% 8 min
clonmocky-p dd1273cde91b dpage/pgadmin 8082:80 0.04% 8 min
clonmocky-a 3665d6a4ed70 firnex/clon-moc 8080:8080 0.16% 8 min
Showing 4 items


localhost:8080/dashboard
Clon Mocky
Panel de Control
Proyectos
Usuarios
Idioma
admin
Panel de Control
Proyectos 1
Mocks 3
Usuarios 2
Proyectos
+ Crear Proyecto
Nombre
Descripción
Cantidad de Mocks
Fecha de Creación
Acciones
API de Ejemplo
Proyecto de demostración con mocks de una API REST de usuarios y productos.
3
27/02/2026 16:39
Mocks


localhost:8082/browser/
pgAdmin
File
Object
Tools
Help
admin@admin.com (Internal)
Object Explorer
Servers (1)
test
Databases
Login/Group Roles
Tablespaces
Dashboard
Properties
SQL
Statistics
Dependencies
Dependents
Processes
Activity
State
Configuration
Logs
System
Server sessions
Total
Active
Idle
Transactions per second
Transactions
Commits
Rollbacks
Tuples in
Inserts
Updates
Deletes
Tuples out
Fetched
Returned
Block I/O
Reads
Hits